CHAPTER 16

Exceptions and Templates

TOPICS

16.1 Exceptions
16.2 Function Templates
16.3 Focus on Software Engineering: Where to Start When Defining Templates
16.4 Class Templates

16.1 Exceptions

CONCEPT: Exceptions are used to signal errors or unexpected events that occur while a program is running.

Error testing is usually a straightforward process involving `if` statements or other control mechanisms. For example, the following code segment will trap a division-by-zero error before it occurs:

```
if (denominator == 0)
     cout << "ERROR: Cannot divide by zero.\n";
else
     quotient = numerator / denominator;
```

But what if similar code is part of a function that returns the quotient, as in the following example?

```
// An unreliable division function
double divide(int numerator, int denominator)
{
     if (denominator == 0)
     {
          cout << "ERROR: Cannot divide by zero.\n";
          return 0;
     }
          else
               return static_cast<double>(numerator) / denominator;
}
```

Functions commonly signal error conditions by returning a predetermined value. Apparently, the function in this example returns 0 when division by zero has been attempted.

This is unreliable, however, because 0 is a valid result of a division operation. Even though the function displays an error message, the part of the program that calls the function will not know when an error has occurred. Problems like these require sophisticated error handling techniques.

Throwing an Exception

One way of handling complex error conditions is with *exceptions*. An exception is a value or an object that signals an error. When the error occurs, an exception is "thrown." For example, the following code shows the `divide` function, modified to throw an exception when division by zero has been attempted:


VideoNote
Throwing an Exception

```
double divide(int numerator, int denominator)
{
     if (denominator == 0)
          throw "ERROR: Cannot divide by zero.\n";
     else
          return static_cast<double>(numerator) / denominator;
}
```

The following statement causes the exception to be thrown:

```
throw "ERROR: Cannot divide by zero.\n";
```

The `throw` key word is followed by an argument, which can be any value. As you will see, the value of the argument will be used to determine the nature of the error. The function above simply throws a string containing an error message.

The line containing a `throw` statement is known as the *throw point*. When a `throw` statement is executed, the function that has thrown the exception aborts its execution and control is passed to another part of the program known as an *exception handler*. The exception handler gracefully responds to the error condition, rather than letting the program abruptly terminate.

Handling an Exception

To handle an exception, a program must have a *try/catch* construct. The general format of the try/catch construct is:

```
try
{
     // code here calls functions or object member
     // functions that might throw an exception.
}
catch(ExceptionParameter)
{
     // code here handles the exception
}
// Repeat as many catch blocks as needed.
```


VideoNote
Handling an Exception

The first part of the construct is the *try block*. This starts with the key word `try` and is followed by a block of code executing any statements that might directly or indirectly cause an exception to be thrown. The try block is immediately followed by one or more *catch blocks*,

which are the exception handlers. A catch block starts with the key word `catch`, followed by a set of parentheses containing the definition of an exception parameter. For example, here is a try/catch construct that can be used with the `divide` function:

```
try
{
     quotient = divide(num1, num2);
     cout << "The quotient is " << quotient << endl;
}
catch (string exceptionString)
{
     cout << exceptionString;
}
```

Because the `divide` function throws an exception whose value is a string, there must be an exception handler that catches a string. The catch block shown catches the error message in the `exceptionString` parameter then displays it with `cout`. Now let's look at an entire program to see how `throw`, `try`, and `catch` work together. In the first sample run of Program 16-1, valid data are given. This shows how the program should run with no errors. In the second sample running, a denominator of 0 is given. This shows the result of the exception being thrown.

Program 16-1

```
1  // This program demonstrates an exception being thrown and caught.
2  #include <iostream>
3  #include <string>
4  using namespace std;
5
6  // Function prototype
7  double divide(int, int);
8
9  int main()
10 {
11     int num1, num2;  // To hold two numbers
12     double quotient; // To hold the quotient of the numbers
13
14     // Get two numbers.
15     cout << "Enter two numbers: ";
16     cin >> num1 >> num2;
17
18     // Divide num1 by num2 and catch any
19     // potential exceptions.
20     try
21     {
22          quotient = divide(num1, num2);
23          cout << "The quotient is " << quotient << endl;
24     }
25     catch (string exceptionString)
26     {
27          cout << exceptionString;
28     }
29
```

(program continues)

Program 16-1 *(continued)*

```
30       cout << "End of the program.\n";
31       return 0;
32   }
33
34   //*****************************************
35   // The divide function divides the numerator *
36   // by the denominator. If the denominator is *
37   // zero, the function throws an exception.   *
38   //*****************************************
39
40   double divide(int numerator, int denominator)
41   {
42       if (denominator == 0)
43       {
44           string exceptionString = "ERROR: Cannot divide by zero.\n";
45           throw exceptionString;
46       }
47
48       return static_cast<double>(numerator) / denominator;
49   }
```

Program Output with Example Input Shown in Bold

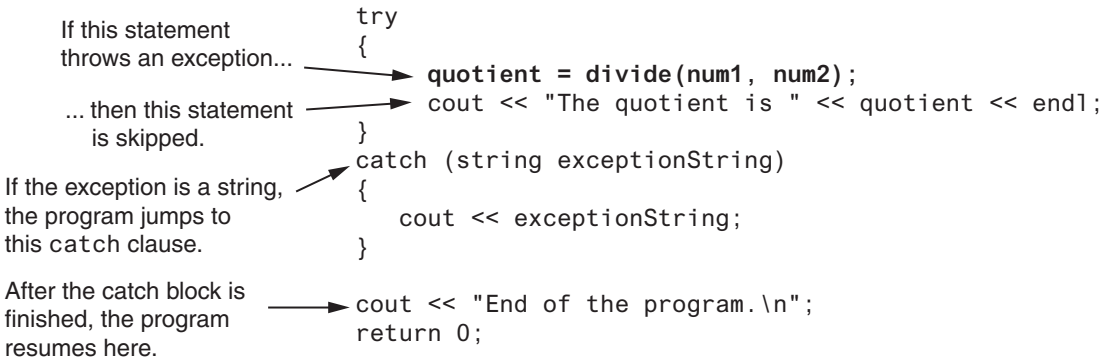
```
Enter two numbers: 12 2 Enter
The quotient is 6
End of the program.
```

Program Output with Different Example Input Shown in Bold

```
Enter two numbers: 12 0 Enter
ERROR: Cannot divide by zero.
End of the program.
```

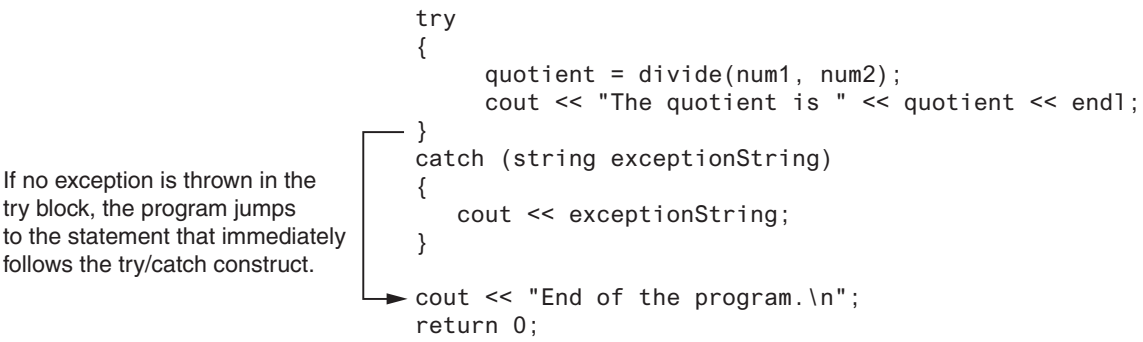
As you can see from the second output screen, the exception caused the program to jump out of the `divide` function and into the catch block. After the catch block has finished, the program resumes with the first statement after the try/catch construct. This is illustrated in Figure 16-1.

Figure 16-1 How the try/catch construct works



In the first output screen, the user entered nonnegative values. No exception was thrown in the try block, so the program skipped the catch block and jumped to the statement immediately following the try/catch construct, which is in line 30. This is illustrated in Figure 16-2.

Figure 16-2 No exception was thrown



What if an Exception Is Not Caught?

There are two possible ways for a thrown exception to go uncaught. The first possibility is for the try/catch construct to contain no catch blocks with an exception parameter of the right data type. The second possibility is for the exception to be thrown from outside a try block. In either case, the exception will cause the entire program to abort execution.

Object-Oriented Exception Handling with Classes

Now that you have an idea of how the exception mechanism in C++ works, we will examine an object-oriented approach to exception handling. Recall the `Rectangle` class that was introduced in Chapter 13. That class had the mutator functions `setWidth` and `setLength` for setting the rectangle's width and length. If a negative value was passed to either of these functions, the class displayed an error message and aborted the program. The following code shows an improved version of the `Rectangle` class. This version throws an exception when a negative value is passed to `setWidth` or `setLength`. (These files can be found in the Student Source Code Folder Chapter 16\Rectangle Version 1.)

Contents of `Rectangle.h` (Version 1)

```
1  // Specification file for the Rectangle class
2  #ifndef RECTANGLE_H
3  #define RECTANGLE_H
4
5  class Rectangle
6  {
7      private:
8          double width;     // The rectangle's width
9          double length;    // The rectangle's length
10     public:
11         // Exception class
12         class NegativeSize
13             { };          // Empty class declaration
14
```

```
15          // Default constructor
16          Rectangle()
17              { width = 0.0; length = 0.0; }
18
19          // Mutator functions, defined in Rectangle.cpp
20          void setWidth(double);
21          void setLength(double);
22
23          // Accessor functions
24          double getWidth() const
25              { return width; }
26
27          double getLength() const
28              { return length; }
29
30          double getArea() const
31              { return width * length; }
32  };
33  #endif
```

Notice the empty class declaration that appears in the public section, in lines 12 and 13. The `NegativeSize` class has no members. The only important part of the class is its name, which will be used in the exception-handling code. Now look at the Rectangle.cpp file, where the `setWidth` and `setLength` member functions are defined.

Contents of `Rectangle.cpp` (Version 1)

```
1   // Implementation file for the Rectangle class.
2   #include "Rectangle.h"
3
4   //*********************************************************
5   // setWidth sets the value of the member variable width.  *
6   //*********************************************************
7
8   void Rectangle::setWidth(double w)
9   {
10      if (w >= 0)
11          width = w;
12      else
13          throw NegativeSize();
14  }
15
16  //************************************************************
17  // setLength sets the value of the member variable length. *
18  //************************************************************
19
20  void Rectangle::setLength(double len)
21  {
22      if (len >= 0)
23          length = len;
24      else
25          throw NegativeSize();
26  }
```

In the `setWidth` function, the parameter `w` is tested by the `if` statement in line 10. If `w` is greater than or equal to 0, its value is assigned to the `width` member variable. If `w` holds a negative number, however, the statement in line 13 is executed.

```
throw NegativeSize();
```

The `throw` statement's argument, `NegativeSize()`, causes an instance of the `NegativeSize` class to be created and thrown as an exception.

The same series of events takes place in the `setLength` function. If the value in the `len` parameter is greater than or equal to 0, its value is assigned to the `length` member variable. If `len` holds a negative number, an instance of the `NegativeSize` class is thrown as an exception in line 25.

This way of reporting errors is much more graceful than simply aborting the program. Any code that uses the `Rectangle` class must simply have a catch block to handle the `NegativeSize` exceptions that the `Rectangle` class might throw. Program 16-2 shows an example. (This file can be found in the Student Source Code Folder Chapter 16\Rectangle Version 1.)

Program 16-2

```
 1  // This program demonstrates Rectangle class exceptions.
 2  #include <iostream>
 3  #include "Rectangle.h"
 4  using namespace std;
 5
 6  int main()
 7  {
 8      double width;
 9      double length;
10
11      // Create a Rectangle object.
12      Rectangle myRectangle;
13
14      // Get the width and length.
15      cout << "Enter the rectangle's width: ";
16      cin >> width;
17      cout << "Enter the rectangle's length: ";
18      cin >> length;
19
20      // Store these values in the Rectangle object.
21      try
22      {
23          myRectangle.setWidth(width);
24          myRectangle.setLength(length);
25          cout << "The area of the rectangle is "
26               << myRectangle.getArea() << endl;
27      }
28      catch (Rectangle::NegativeSize)
29      {
30          cout << "Error: A negative value was entered.\n";
31      }
```

(program continues)

Program 16-2 *(continued)*

```
32       cout << "End of the program.\n";
33
34       return 0;
35   }
```

Program Output with Example Input Shown in Bold

```
Enter the rectangle's width: 10 Enter
Enter the rectangle's length: 20 Enter
The area of the rectangle is 200
End of the program.
```

Program Output with Different Example Input Shown in Bold

```
Enter the rectangle's width: 5 Enter
Enter the rectangle's length: -5 Enter
Error: A negative value was entered.
End of the program.
```

The `catch` statement in line 28 catches the `NegativeSize` exception when it is thrown by any of the statements in the try block. Inside the `catch` statement's parentheses is the name of the `NegativeSize` class. Because the `NegativeSize` class is declared inside the `Rectangle` class, we have to fully qualify the class name with the scope resolution operator.

Notice we did not define a parameter of the `NegativeSize` class in the `catch` statement. In this case, the `catch` statement only needs to specify the type of exception it handles.

Multiple Exceptions

The programs we have studied so far test only for a single type of error and throw only a single type of exception. In many cases, a program will need to test for several different types of errors and signal which one has occurred. C++ allows you to throw and catch multiple exceptions. The only requirement is that each different exception be of a different type. You then code a separate catch block for each type of exception that may be thrown in the try block.

For example, suppose we wish to expand the `Rectangle` class so it throws one type of exception when a negative value is specified for the `width`, and another type of exception when a negative value is specified for the `length`. First, we declare two different exception classes, such as:

```
// Exception class for a negative width
class NegativeWidth
    { };

// Exception class for a negative length
class NegativeLength
    { };
```

An instance of `NegativeWidth` will be thrown when a negative value is specified for the `width`, and an instance of `NegativeLength` will be thrown when a negative value is

specified for the `length`. The code for the modified `Rectangle` class is shown here. (These files can be found in the Student Source Code Folder Chapter 16\Rectangle Version 2.)

Contents of `Rectangle.h` (Version 2)

```
// Specification file for the Rectangle class
#ifndef RECTANGLE_H
#define RECTANGLE_H

class Rectangle
{
    private:
        double width;      // The rectangle's width
        double length;     // The rectangle's length
    public:
        // Exception class for a negative width
        class NegativeWidth
            { };

        // Exception class for a negative length
        class NegativeLength
            { };

        // Default constructor
        Rectangle()
            { width = 0.0; length = 0.0; }

        // Mutator functions, defined in Rectangle.cpp
        void setWidth(double);
        void setLength(double);

        // Accessor functions
        double getWidth() const
            { return width; }

        double getLength() const
            { return length; }

        double getArea() const
            { return width * length; }
};
#endif
```

Contents of `Rectangle.cpp` (Version 2)

```
// Implementation file for the Rectangle class.
#include "Rectangle.h"

//***********************************************************
// setWidth sets the value of the member variable width.    *
//***********************************************************

```

```
 8  void Rectangle::setWidth(double w)
 9  {
10      if (w >= 0)
11          width = w;
12      else
13          throw NegativeWidth();
14  }
15
16  //*********************************************************
17  // setLength sets the value of the member variable length. *
18  //*********************************************************
19
20  void Rectangle::setLength(double len)
21  {
22      if (len >= 0)
23          length = len;
24      else
25          throw NegativeLength();
26  }
```

Notice in the definition of the `setWidth` function (in Rectangle.cpp) that an instance of the `NegativeWidth` class is thrown in line 13. In the definition of the `setLength` function, an instance of the `NegativeLength` class is thrown in line 25. Program 16-3 demonstrates this class. (This file can be found in the Student Source Code Folder Chapter 16\Rectangle Version 2.)

Program 16-3

```
 1  // This program demonstrates Rectangle class exceptions.
 2  #include <iostream>
 3  #include "Rectangle.h"
 4  using namespace std;
 5
 6  int main()
 7  {
 8      double width;
 9      double length;
10
11      // Create a Rectangle object.
12      Rectangle myRectangle;
13
14      // Get the width and length.
15      cout << "Enter the rectangle's width: ";
16      cin >> width;
17      cout << "Enter the rectangle's length: ";
18      cin >> length;
19
20      // Store these values in the Rectangle object.
21      try
22      {
23          myRectangle.setWidth(width);
```

```
24           myRectangle.setLength(length);
25           cout << "The area of the rectangle is "
26                << myRectangle.getArea() << endl;
27      }
28      catch (Rectangle::NegativeWidth)
29      {
30           cout << "Error: A negative value was given "
31                << "for the rectangle's width.\n";
32      }
33      catch (Rectangle::NegativeLength)
34      {
35           cout << "Error: A negative value was given "
36                << "for the rectangle's length.\n";
37      }
38
39      cout << "End of the program.\n";
40      return 0;
41  }
```

Program Output with Example Input Shown in Bold

```
Enter the rectangle's width: 10 Enter
Enter the rectangle's length: 20 Enter
The area of the rectangle is 200
End of the program.
```

Program Output with Different Example Input Shown in Bold

```
Enter the rectangle's width: -5 Enter
Enter the rectangle's length: 5 Enter
Error: A negative value was given for the rectangle's width.
End of the program.
```

Program Output with Different Example Input Shown in Bold

```
Enter the rectangle's width: 5 Enter
Enter the rectangle's length: -5 Enter
Error: A negative value was given for the rectangle's length.
End of the program.
```

The try block, in lines 21 through 27, contains code that can throw two different types of exceptions. The statement in line 23 can potentially throw a `NegativeWidth` exception, and the statement in line 24 can potentially throw a `NegativeLength` exception. To handle each of these types of exception, there are two `catch` statements. The statement in line 28 catches `NegativeWidth` exceptions, and the statement in line 33 catches `NegativeLength` exceptions.

When an exception is thrown by code in the try block, C++ searches the try/catch construct for a `catch` statement that can handle the exception. If the construct contains a `catch` statement that is compatible with the exception, control of the program is passed to the catch block.

Using Exception Handlers to Recover from Errors

Program 16-3 demonstrates how a try/catch construct can have several `catch` statements in order to handle different types of exceptions. However, the program does not use the exception handlers to recover from any of the errors. When the user enters a negative value for either the width or the length, this program still halts. Program 16-4 shows a better example of effective exception handling. It attempts to recover from the exceptions and get valid data from the user. (This file can be found in the Student Source Code Folder Chapter 16\Rectangle Version 2.)

Program 16-4

```
// This program handles the Rectangle class exceptions.
#include <iostream>
#include "Rectangle.h"
using namespace std;

int main()
{
    double width;           // Rectangle's width
    double length;          // Rectangle's length
    bool tryAgain = true;   // Flag to reread input

    // Create a Rectangle object.
    Rectangle myRectangle;

    // Get the rectangle's width.
    cout << "Enter the rectangle's width: ";
    cin >> width;

    // Store the width in the myRectangle object.
    while (tryAgain)
    {
        try
        {
            myRectangle.setWidth(width);
            // If no exception was thrown, then the
            // next statement will execute.
            tryAgain = false;
        }
        catch (Rectangle::NegativeWidth)
        {
            cout << "Please enter a nonnegative value: ";
            cin >> width;
        }
    }

    // Get the rectangle's length.
    cout << "Enter the rectangle's length: ";
    cin >> length;

```

```
40      // Store the length in the myRectangle object.
41      tryAgain = true;
42      while (tryAgain)
43      {
44          try
45          {
46              myRectangle.setLength(length);
47              // If no exception was thrown, then the
48              // next statement will execute.
49              tryAgain = false;
50          }
51          catch (Rectangle::NegativeLength)
52          {
53              cout << "Please enter a nonnegative value: ";
54              cin >> length;
55          }
56      }
57
58      // Display the area of the rectangle.
59      cout << "The rectangle's area is "
60           << myRectangle.getArea() << endl;
61      return 0;
62  }
```

Program Output with Example Input Shown in Bold

```
Enter the rectangle's width: -1 Enter
Please enter a nonnegative value: 10 Enter
Enter the rectangle's length: -5 Enter
Please enter a nonnegative value: 50 Enter
The rectangle's area is 500
```

Let's look at how this program recovers from a `NegativeWidth` exception. In line 10, a `bool` flag variable, `tryAgain`, is defined and initialized with the value `true`. This variable will indicate whether we need to get a value from the user again. Lines 16 and 17 prompt the user to enter the rectangle's width. Then, the program enters the `while` loop in lines 20 through 34. The loop repeats as long as `tryAgain` is `true`. Inside the loop, the `Rectangle` class's `setWidth` member function is called in line 24. This statement is in a try block. If a `NegativeWidth` exception is thrown, the program will jump to the `catch` statement in line 29. In the `catch` block that follows, the user is asked to enter a nonnegative number. The program then jumps out of the try/catch construct. Because `tryAgain` is still `true`, the loop will repeat.

If a nonnegative number is passed to the `setWidth` member function in line 24, no exception will be thrown. In that case, the statement in line 27 will execute, which sets `tryAgain` to `false`. The program then jumps out of the try/catch construct. Because `tryAgain` is now `false`, the loop will not repeat.

The same strategy is used in lines 37 through 56 to get and validate the rectangle's length.

Extracting Data from the Exception Class

Sometimes we might want an exception object to pass data back to the exception handler. For example, suppose we would like the `Rectangle` class not only to signal when a negative value has been given, but also to pass the value back. This can be accomplished by giving the exception class members in which data can be stored.

In our next modification of the `Rectangle` class, the `NegativeWidth` and `NegativeLength` classes have been expanded, each with a member variable and a constructor. Here is the code for the `NegativeWidth` class:

```
class NegativeWidth
{
private:
     double value;
public:
     NegativeWidth(double val)
          { value = val; }

     double getValue() const
          { return value; }
};
```

When we throw this exception, we want to pass the invalid value as an argument to the class's constructor. This is done in the `setWidth` member function with the following statement:

```
throw NegativeWidth(w);
```

This `throw` statement creates an instance of the `NegativeWidth` class and passes a copy of the `w` variable to the constructor. The constructor then stores this number in `NegativeWidth`'s member variable, `value`. The class instance carries this member variable to the catch block that intercepts the exception.

In the catch block, the value is extracted with code such as

```
catch (Rectangle::NegativeWidth e)
{
     cout << "Error: " << e.getValue()
          << " is an invalid value for the"
          << " rectangle's width.\n";
}
```

Notice the catch block defines a parameter object named `e`. This is necessary because we want to call the class's `getValue` function to retrieve the value that caused the exception.

Here is the code for the `NegativeLength` class:

```
class NegativeLength
{
private:
     double value;
public:
     NegativeLength(double val)
          { value = val; }

     double getValue() const
          { return value; }
};
```

This class also has a member variable named `value`, and a constructor that initializes the variable. When we throw this exception, we follow the same general steps that were just described for the `NegativeWidth` exception. The complete code for the revised `Rectangle` class is shown here. Program 16-5 demonstrates these classes. (These files can be found in the Student Source Code Folder Chapter 16\Rectangle Version 3.)

Contents of `Rectangle.h` (Version 3)

```
// Specification file for the Rectangle class
#ifndef RECTANGLE_H
#define RECTANGLE_H

class Rectangle
{
    private:
        double width;      // The rectangle's width
        double length;     // The rectangle's length
    public:
        // Exception class for a negative width
        class NegativeWidth
        {
        private:
            double value;
        public:
            NegativeWidth(double val)
                { value = val; }

            double getValue() const
                { return value; }
        };

        // Exception class for a negative length
        class NegativeLength
        {
        private:
            double value;
        public:
            NegativeLength(double val)
                { value = val; }

            double getValue() const
                { return value; }
        };

        // Default constructor
        Rectangle()
            { width = 0.0; length = 0.0; }

        // Mutator functions, defined in Rectangle.cpp
        void setWidth(double);
        void setLength(double);

```

```
45          // Accessor functions
46          double getWidth() const
47              { return width; }
48
49          double getLength() const
50              { return length; }
51
52          double getArea() const
53              { return width * length; }
54  };
55  #endif
```

Contents of Rectangle.cpp (Version 3)

```
 1  // Implementation file for the Rectangle class.
 2  #include "Rectangle.h"
 3
 4  //*************************************************************
 5  // setWidth sets the value of the member variable width.   *
 6  //*************************************************************
 7
 8  void Rectangle::setWidth(double w)
 9  {
10      if (w >= 0)
11          width = w;
12      else
13          throw NegativeWidth(w);
14  }
15
16  //*************************************************************
17  // setLength sets the value of the member variable length. *
18  //*************************************************************
19
20  void Rectangle::setLength(double len)
21  {
22      if (len >= 0)
23          length = len;
24      else
25          throw NegativeLength(len);
26  }
```

Program 16-5

```
 1  // This program demonstrates Rectangle class exceptions.
 2  #include <iostream>
 3  #include "Rectangle.h"
 4  using namespace std;
 5
 6  int main()
 7  {
 8      double width;
 9      double length;
10
```

```
11      // Create a Rectangle object.
12      Rectangle myRectangle;
13
14      // Get the width and length.
15      cout << "Enter the rectangle's width: ";
16      cin >> width;
17      cout << "Enter the rectangle's length: ";
18      cin >> length;
19
20      // Store these values in the Rectangle object.
21      try
22      {
23          myRectangle.setWidth(width);
24          myRectangle.setLength(length);
25          cout << "The area of the rectangle is "
26               << myRectangle.getArea() << endl;
27      }
28      catch (Rectangle::NegativeWidth e)
29      {
30          cout << "Error: " << e.getValue()
31               << " is an invalid value for the"
32               << " rectangle's width.\n";
33      }
34      catch (Rectangle::NegativeLength e)
35      {
36          cout << "Error: " << e.getValue()
37               << " is an invalid value for the"
38               << " rectangle's length.\n";
39      }
40
41      cout << "End of the program.\n";
42      return 0;
43  }
```

Program Output with Example Input Shown in Bold

```
Enter the rectangle's width: -1 [Enter]
Enter the rectangle's length: 10 [Enter]
Error: -1 is an invalid value for the rectangle's width.
End of the program.
```

Program Output with Different Example Input Shown in Bold

```
Enter the rectangle's width: 5 [Enter]
Enter the rectangle's length: -1 [Enter]
Error: -1 is an invalid value for the rectangle's length.
End of the program.
```

Unwinding the Stack

Once an exception has been thrown, the program cannot jump back to the throw point. The function that executes a `throw` statement will immediately terminate. If that function was called by another function, and the exception is not caught, then the calling

function will terminate as well. This process, known as *unwinding the stack*, continues for the entire chain of nested function calls, from the throw point, all the way back to the try block.

Rethrowing an Exception

It is possible for try blocks to be nested. For example, look at this code segment:

```
try
{
    doSomething();
}
catch(exception1)
{
    // code to handle exception 1
}
catch(exception2)
{
    // code to handle exception 2
}
```

In this try block, the function `doSomething` is called. There are two catch blocks, one that handles `exception1`, and another that handles `exception2`. If the `doSomething` function also has a try block, then it is nested inside the one shown.

With nested try blocks, it is sometimes necessary for an inner exception handler to pass an exception to an outer exception handler. Sometimes both an inner and an outer catch block must perform operations when a particular exception is thrown. These situations require that the inner catch block *rethrow* the exception so the outer catch block has a chance to catch it.

A catch block can rethrow an exception with the `throw` statement. For example, suppose the `doSomething` function (called in the throw block above) calls the `doSomethingElse` function, which potentially can throw `exception1` or `exception3`. Suppose `doSomething` does not want to handle `exception1`. Instead, it wants to rethrow it to the outer block. The following code segment illustrates how this is done:

```
try
{
    doSomethingElse();
}
catch(exception1)
{
    throw; // Rethrow the exception
}
catch(exception3)
{
    // Code to handle exception 3
}
```

When the first catch block catches `exception1`, the `throw` statement simply throws the exception again. The catch block in the outer try/catch construct will then handle the exception.

Handling the bad_alloc Exception

Recall from Chapter 9 that when the new operator fails to allocate memory, an exception is thrown. Now that you've seen how to handle exceptions, you can write code that determines whether the new operator was successful.

When the new operator fails to allocate memory, C++ throws a bad_alloc exception. The bad_alloc exception type is defined in the <new> header file, so any program that attempts to catch this exception should have the following directive:

```
#include <new>
```

The bad_alloc exception is in the std namespace, so be sure to have the using namespace std; statement in your code as well.

Here is the general format of a try/catch construct that catches the bad_alloc exception:

```
try
{
    // Code that uses the new operator
}
catch (bad_alloc)
{
    // Code that responds to the error
}
```

Program 16-6 shows an example. The program uses the new operator to allocate a 10,000-element array of doubles. If the new operator fails, an error message is displayed.

Program 16-6

```
1  // This program demonstrates the bad_alloc exception.
2  #include <iostream>
3  #include <new>          // Needed for bad_alloc
4  using namespace std;
5
6  int main()
7  {
8      double *ptr = nullptr; // Pointer to double
9
10     try
11     {
12         ptr = new double [10000];
13     }
14     catch (bad_alloc)
15     {
16         cout << "Insufficient memory.\n";
17     }
18
19     return 0;
20 }
```

Using the noexcept Key Word

Beginning with C++ 11, you can declare a function with the `noexcept` specifier to indicate that the function does not throw an exception and does not call any other functions that throw an exception. Here is an example:

```
void sayHello() noexcept
{
   cout << "Hello world" << endl;
}
```

The primary purpose of the `noexcept` key word is to help the compiler produce faster code. We won't discuss the details here, but when the compiler knows that a function cannot cause an uncaught exception to be thrown, it can optimize the executable code that it generates.

The `noexcept` key word is not meant to guarantee that no exceptions will be thrown, however. For example, in the following code, `myFunction` is declared as `noexcept`. It calls `anotherFunction`, which may or may not throw an exception. When you declare a function as `noexcept`, the compiler does not trace through the code to make sure that no exceptions are possible. If an exception is thrown from within a function that is declared as `noexcept`, the program's execution will be terminated.

```
void myFunction() noexcept
{
   anotherFuntion();
}
```

You can also use the `noexcept` key word to specify that a function does not throw an exception under specific conditions. Here is the general format:

```
noexcept(BooleanExpression)
```

This general format specifies that the function does not throw an exception when the *`BooleanExpression`* is true. Here is an example:

```
double divide(int numerator, int denominator) noexcept(denominator != 0)
{
   return static_cast<double>(numerator) / denominator;
}
```

This code defines a function named `divide` that has two `int` parameters: `numerator` and `denominator`. The `noexcept` clause specifies that the function will not throw an exception when `denominator` is not equal to 0.



Checkpoint

16.1 What is the difference between a try block and a catch block?

16.2 What happens if an exception is thrown, but not caught?

16.3 If multiple exceptions can be thrown, how does the catch block know which exception to catch?

16.4 After the catch block has handled the exception, where does program execution resume?

16.5 How can an exception pass data back to the exception handler?

16.2 Function Templates

CONCEPT: **A function template is a "generic" function that can work with any data type. The programmer writes the specifications of the function, but substitutes parameters for data types. When the compiler encounters a call to the function, it generates code to handle the specific data type(s) used in the call.**

Introduction

Overloaded functions make programming convenient because only one function name must be remembered for a set of functions that perform similar operations. Each of the functions, however, must still be written individually, even if they perform the same operation. For example, suppose a program uses the following overloaded `square` functions:

```
int square(int number)
{
    return number * number;
}
double square(double number)
{
    return number * number;
}
```

The only differences between these two functions are the data types of their return values and their parameters. In situations like this, it is more convenient to write a *function template* than an overloaded function. Function templates allow you to write a single function definition that works with many different data types, instead of having to write a separate function for each data type used.

A function template is not an actual function, but a "mold" the compiler uses to generate one or more functions. When writing a function template, you do not have to specify actual types for the parameters, return value, or local variables. Instead, you use a *type parameter* to specify a generic data type. When the compiler encounters a call to the function, it examines the data types of its arguments and generates the function code that will work with those data types. (The generated code is known as a *template function*.)

Here is a function template for the `square` function:



VideoNote
Writing a Function Template

```
template <typename T>
T square(T number)
{
    return number * number;
}
```

The beginning of a function template is marked by a *template prefix*, which begins with the key word `template`. Next is a set of angled brackets that contains one or more generic data types used in the template. A generic data type starts with the key word `typename` followed by a parameter name that stands for the data type. The example just given only uses one, which is named `T`. (If there were more, they would be separated by commas.) After this, the function definition is written as usual, except the type parameters are substituted for the actual data type names. In the example, the function header reads

```
T square(T number)
```

T is the type parameter, or generic data type. The header defines square as a function that returns a value of type T and uses a parameter, number, which is also of type T. As mentioned before, the compiler examines each call to square and fills in the appropriate data type for T. For example, the following call uses an int argument:

```
int y, x = 4;
y = square(x);
```

This code will cause the compiler to generate the function

```
int square(int number)
{
    return number * number;
}
```

while the following statements

```
double y, f = 6.2
y = square(f);
```

will generate the function

```
double square(double number)
{
    return number * number;
}
```

Program 16-7 demonstrates how this function template is used.

Program 16-7

```
1  // This program uses a function template.
2  #include <iostream>
3  #include <iomanip>
4  using namespace std;
5
6  // Template definition for square function.
7  template <typename T>
8  T square(T number)
9  {
10     return number * number;
11 }
12
13 int main()
14 {
15     int userInt;       // To hold integer input
16     double userDouble; // To hold double input
17
18     cout << setprecision(5);
19     cout << "Enter an integer and a floating-point value: ";
20     cin >> userInt >> userDouble;
21     cout << "Here are their squares: ";
22     cout << square(userInt) << " and "
```

```
23           << square(userDouble) << endl;
24      return 0;
25  }
```

Program Output with Example Input Shown in Bold

```
Enter an integer and a floating-point value: 12 4.2 Enter
Here are their squares: 144 and 17.64
```



NOTE: All type parameters defined in a function template must appear at least once in the function parameter list.



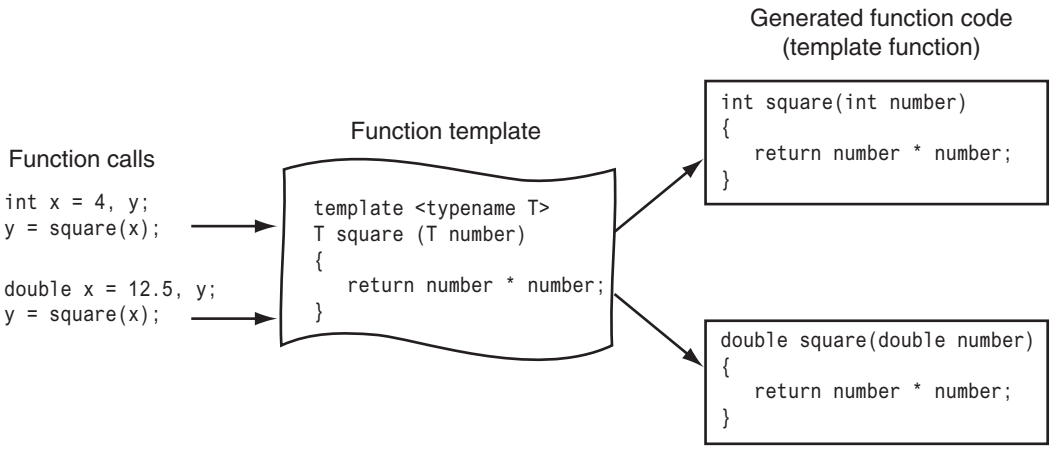
NOTE : When declaring a generic type in a template, the word `class` may be used instead of the word `typename`. For example, the template for the `square` function could be written like this:

```
template <class T>
T square(T number)
{
   return number * number;
}
```

Either word, `typename` or `class`, is correct in this usage. Most programmers prefer `typename` so we will continue to use it in this book.

Because the compiler encountered two calls to `square` in Program 16-7, each with a different parameter type, it generated the code for two instances of the function: one with an `int` parameter and `int` return type, the other with a `double` parameter and `double` return type. This is illustrated in Figure 16-3.

Figure 16-3 Generation of template functions



Notice in Program 16-7 that the template appears before all calls to `square`. As with regular functions, the compiler must already know the template's contents when it encounters a call to the template function. Templates, therefore, should be placed near the top of the program or in a header file.



NOTE: A function template is merely the specification of a function and by itself does not cause memory to be used. An actual instance of the function is created in memory when the compiler encounters a call to the template function.

Program 16-8 shows another example of a function template. The function, `swapVars`, uses two references to type `T` as parameters. The function swaps the contents of the variables referenced by the parameters.

Program 16-8

```
1  // This program demonstrates the swapVars function template.
2  #include <iostream>
3  using namespace std;
4
5  template <typename T>
6  void swapVars(T &var1, T &var2)
7  {
8      T temp;
9
10     temp = var1;
11     var1 = var2;
12     var2 = temp;
13 }
14
15 int main()
16 {
17     char firstChar, secondChar;       // Two chars
18     int firstInt, secondInt;          // Two ints
19     double firstDouble, secondDouble; // Two doubles
20
21     // Get and swapVars two chars
22     cout << "Enter two characters: ";
23     cin >> firstChar >> secondChar;
24     swapVars(firstChar, secondChar);
25     cout << firstChar << " " << secondChar << endl;
26
27     // Get and swapVars two ints
28     cout << "Enter two integers: ";
29     cin >> firstInt >> secondInt;
30     swapVars(firstInt, secondInt);
31     cout << firstInt << " " << secondInt << endl;
32
33     // Get and swapVars two doubles
34     cout << "Enter two floating-point numbers: ";
```

```
35      cin >> firstDouble >> secondDouble;
36      swapVars(firstDouble, secondDouble);
37      cout << firstDouble << " " << secondDouble << endl;
38      return 0;
39  }
```

Program Output with Example Input Shown in Bold

```
Enter two characters: A B Enter
B A
Enter two integers: 5 10 Enter
10 5
Enter two floating-point numbers: 1.2 9.6 Enter
9.6 1.2
```

Using Operators in Function Templates

The `square` template shown earlier uses the `*` operator with the `number` parameter. This works well as long as `number` is of a primitive data type such as `int`, `float`, and so on. If a user-defined class object is passed to the `square` function, however, the class must contain code for an overloaded `*` operator. If not, the compiler will generate a function with an error.

Always remember a class object passed to a function template must support all the operations the function will perform on the object. For instance, if the function performs a comparison on the object (with `>`, `<`, `==`, or another relational operator), those operators must be overloaded by the class object.

Function Templates with Multiple Types

More than one generic type may be used in a function template. Each type must have its own parameter, as shown in Program 16-9. This program uses a function template named `larger`. This template uses two type parameters: `T1` and `T2`. The sizes of the function parameters, `var1` and `var2`, are compared, and the function returns the number of bytes occupied by the larger of the two. Because the function parameters are specified with different types, the function generated from this template can accept two arguments of different types.

Program 16-9

```
 1  // This program demonstrates a function template
 2  // with two type parameters.
 3  #include <iostream>
 4  using namespace std;
 5
 6  template <typename T1, typename T2>
 7  int largest(const T1 &var1, T2 &var2)
 8  {
 9      if (sizeof(var1) > sizeof(var2))
10          return sizeof(var1);
11      else
```

(program continues)

Program 16-9 *(continued)*

```
12          return sizeof(var2);
13  }
14
15  int main()
16  {
17      int i = 0;
18      char c = ' ';
19      float f = 0.0;
20      double d = 0.0;
21
22      cout << "Comparing an int and a double, the largest\n"
23           << "of the two is " << largest(i, d) << " bytes.\n";
24
25      cout << "Comparing a char and a float, the largest\n"
26           << "of the two is " << largest(c, f) << " bytes.\n";
27
28      return 0;
29  }
```

Program Output

```
Comparing an int and a double, the largest
of the two is 8 bytes.
Comparing a char and a float, the largest
of the two is 4 bytes.
```



NOTE: Each type parameter declared in the template prefix must appear at least once in the function parameter list.

Overloading with Function Templates

Function templates may be overloaded. As with regular functions, function templates are overloaded by having different parameter lists. For example, there are two overloaded versions of the `sum` function in Program 16-10. The first version accepts two arguments, and the second version accepts three.

Program 16-10

```
1  // This program demonstrates an overloaded function template.
2  #include <iostream>
3  using namespace std;
4
5  template <typename T>
6  T sum(T val1, T val2)
7  {
8      return val1 + val2;
9  }
```

```
10
11  template <typename T>
12  T sum(T val1, T val2, T val3)
13  {
14      return val1 + val2 + val3;
15  }
16
17  int main()
18  {
19      double num1, num2, num3;
20
21      // Get two values and display their sum.
22      cout << "Enter two values: ";
23      cin >> num1 >> num2;
24      cout << "Their sum is " << sum(num1, num2) << endl;
25
26      // Get three values and display their sum.
27      cout << "Enter three values: ";
28      cin >> num1 >> num2 >> num3;
29      cout << "Their sum is " << sum(num1, num2, num3) << endl;
30      return 0;
31  }
```

Program Output with Example Input Shown in Bold

```
Enter two values: 12.5 6.9 Enter
Their sum is 19.4
Enter three values: 45.76 98.32 10.51 Enter
Their sum is 154.59
```

There are other ways to perform overloading with function templates as well. For example, a program might contain a regular (nontemplate) version of a function as well as a template version. As long as each has a different parameter list, they can coexist as overloaded functions.

16.3 Focus on Software Engineering: Where to Start When Defining Templates

Quite often, it is easier to convert an existing function into a template than to write a template from scratch. With this in mind, you should start designing a function template by writing it first as a regular function. For example, the `swapVars` template in Program 16-8 would have been started as something like the following:

```
void swapVars(int &var1, int &var2)
{
    int temp;

    temp = var1;
    var1 = var2;
    var2 = temp;
}
```

Once this function is properly tested and debugged, converting it to a template is a simple process. First, the `template <typename T>` header is added, then all the references to `int` that must be changed are replaced with the data type parameter `T`.



Checkpoint

16.6 When does the compiler actually generate code for a function template?

16.7 The following function accepts an `int` argument and returns half of its value as a `double`:

```
double half(int number)
{
       return number / 2.0;
}
```

Write a template that will implement this function to accept an argument of any type.

16.8 What must you be sure of when passing a class object to a function template that uses an operator, such as * or >?

16.9 What is the best method for writing a function template?

16.4 Class Templates

CONCEPT: Templates may also be used to create generic classes and abstract data types. Class templates allow you to create one general version of a class without having to duplicate code to handle multiple data types.

Recall the `IntArray` class from Chapter 14. By overloading the `[]` operator, this class allows you to implement `int` arrays that perform bounds checking. But suppose you would like to have a version of this class for other data types. Of course, you could design specialized classes such as `LongArray`, `FloatArray`, `DoubleArray`, and so forth. A better solution, however, is to design a single class template that works with any primitive data type. In this section, we will convert the `IntArray` class into a generalized template named `SimpleVector`.

Declaring a class template is very similar to declaring a function template. First, a template prefix, such as `template <typename T>`, is placed before the class declaration. As with function templates, `T` (or whatever identifier you choose to use) is a data type parameter. Then, throughout the class declaration, the data type parameter is used where you wish to support any data type. Below is the `SimpleVector` class template declaration.

Contents of `SimpleVector.h`

```
1  // SimpleVector class template
2  #ifndef SIMPLEVECTOR_H
3  #define SIMPLEVECTOR_H
4  #include <iostream>
```

```
#include <new>      // Needed for bad_alloc exception
#include <cstdlib>  // Needed for the exit function
using namespace std;

template <typename T>
class SimpleVector
{
private:
    T *aptr;         // To point to the allocated array
    int arraySize;   // Number of elements in the array
    void memError(); // Handles memory allocation errors
    void subError(); // Handles subscripts out of range

public:
    // Default constructor
    SimpleVector()
        { aptr = 0; arraySize = 0;}

    // Constructor declaration
    SimpleVector(int);

    // Copy constructor declaration
    SimpleVector(const SimpleVector &);

    // Destructor declaration
    ~SimpleVector();

    // Accessor to return the array size
    int size() const
        { return arraySize; }

    // Accessor to return a specific element
    T getElementAt(int position);

    // Overloaded [] operator declaration
    T &operator[](const int &);
};

//*********************************************************
// Constructor for SimpleVector class. Sets the size of the *
// array and allocates memory for it.                       *
//*********************************************************

template <typename T>
SimpleVector<T>::SimpleVector(int s)
{
    arraySize = s;
    // Allocate memory for the array.
    try
    {
        aptr = new T [s];
    }
```

```
57      catch (bad_alloc)
58      {
59          memError();
60      }
61
62      // Initialize the array.
63      for (int count = 0; count < arraySize; count++)
64          *(aptr + count) = 0;
65  }
66
67  //*****************************************
68  // Copy Constructor for SimpleVector class. *
69  //*****************************************
70
71  template <typename T>
72  SimpleVector<T>::SimpleVector(const SimpleVector &obj)
73  {
74      // Copy the array size.
75      arraySize = obj.arraySize;
76
77      // Allocate memory for the array.
78      aptr = new T [arraySize];
79      if (aptr == 0)
80          memError();
81
82      // Copy the elements of obj's array.
83      for(int count = 0; count < arraySize; count++)
84          *(aptr + count) = *(obj.aptr + count);
85  }
86
87  //*************************************
88  // Destructor for SimpleVector class.  *
89  //*************************************
90
91  template <typename T>
92  SimpleVector<T>::~SimpleVector()
93  {
94      if (arraySize > 0)
95          delete [] aptr;
96  }
97
98  //*********************************************************
99  // memError function. Displays an error message and      *
100 // terminates the program when memory allocation fails.  *
101 //*********************************************************
102
103 template <typename T>
104 void SimpleVector<T>::memError()
105 {
106     cout << "ERROR:Cannot allocate memory.\n";
107     exit(EXIT_FAILURE);
108 }
```

```
109
110 //*************************************************************
111 // subError function. Displays an error message and          *
112 // terminates the program when a subscript is out of range. *
113 //*************************************************************
114
115 template <typename T>
116 void SimpleVector<T>::subError()
117 {
118     cout << "ERROR: Subscript out of range.\n";
119     exit(EXIT_FAILURE);
120 }
121
122 //*******************************************************
123 // getElementAt function. The argument is a subscript.  *
124 // This function returns the value stored at the        *
125 // subcript in the array.                               *
126 //*******************************************************
127
128 template <typename T>
129 T SimpleVector<T>::getElementAt(int sub)
130 {
131     if (sub < 0 || sub >= arraySize)
132         subError();
133     return aptr[sub];
134 }
135
136 //*******************************************************
137 // Overloaded [] operator. The argument is a subscript.  *
138 // This function returns a reference to the element      *
139 // in the array indexed by the subscript.               *
140 //*******************************************************
141
142 template <typename T>
143 T &SimpleVector<T>::operator[](const int &sub)
144 {
145     if (sub < 0 || sub >= arraySize)
146         subError();
147     return aptr[sub];
148 }
149 #endif
```



NOTE: The `arraySize` member variable is declared as an `int`. This is because it holds the size of the array, which will be an integer value, regardless of the data type of the array. This is also why the `size` member function returns an `int`.

Defining Objects of the Class Template

Class template objects are defined like objects of ordinary classes, with one small difference: the data type you wish to pass to the type parameter must be specified. Placing the data type name inside angled brackets immediately following the class name does this. For example, the following statements create two `SimpleVector` objects: `intTable` and `doubleTable`.

```
SimpleVector<int> intTable(10);
SimpleVector<double> doubleTable(10);
```

In the definition of `intTable`, the data type `int` will be used in the template everywhere the type parameter `T` appears. This will cause `intTable` to store an array of `int`s. Likewise, the definition of `doubleTable` passes the data type `double` into the parameter `T`, causing it to store an array of `double`s. This is demonstrated in Program 16-11.

Program 16-11

```
1  // This program demonstrates the SimpleVector template.
2  #include <iostream>
3  #include "SimpleVector.h"
4  using namespace std;
5
6  int main()
7  {
8      const int SIZE = 10; // Number of elements
9      int count;           // Loop counter
10
11     // Create a SimpleVector of ints.
12     SimpleVector<int> intTable(SIZE);
13
14     // Create a SimpleVector of doubles.
15     SimpleVector<double> doubleTable(SIZE);
16
17     // Store values in the two SimpleVectors.
18     for (count = 0; count < SIZE; count++)
19     {
20         intTable[count] = (count * 2);
21         doubleTable[count] = (count * 2.14);
22     }
23
24     // Display the values in the SimpleVectors.
25     cout << "These values are in intTable:\n";
26     for (count = 0; count < SIZE; count++)
27         cout << intTable[count] << " ";
28     cout << endl;
29     cout << "These values are in doubleTable:\n";
30     for (count = 0; count < SIZE; count++)
31         cout << doubleTable[count] << " ";
32     cout << endl;
33
34     // Use the standard + operator on the elements.
35     cout << "\nAdding 5 to each element of intTable"
36          << " and doubleTable.\n";
37     for (count = 0; count < SIZE; count++)
```

```
38      {
39          intTable[count] = intTable[count] + 5;
40          doubleTable[count] = doubleTable[count] + 5.0;
41      }
42
43      // Display the values in the SimpleVectors.
44      cout << "These values are in intTable:\n";
45      for (count = 0; count < SIZE; count++)
46          cout << intTable[count] << " ";
47      cout << endl;
48      cout << "These values are in doubleTable:\n";
49      for (count = 0; count < SIZE; count++)
50          cout << doubleTable[count] << " ";
51      cout << endl;
52
53      // Use the standard ++ operator on the elements.
54      cout << "\nIncrementing each element of intTable and"
55           << " doubleTable.\n";
56      for (count = 0; count < SIZE; count++)
57      {
58          intTable[count]++;
59          doubleTable[count]++;
60      }
61
62      // Display the values in the SimpleVectors.
63      cout << "These values are in intTable:\n";
64      for (count = 0; count < SIZE; count++)
65          cout << intTable[count] << " ";
66      cout << endl;
67      cout << "These values are in doubleTable:\n";
68      for (count = 0; count < SIZE; count++)
69          cout << doubleTable[count] << " ";
70      cout << endl;
71
72      return 0;
73  }
```

Program Output

```
These values are in intTable:
0 2 4 6 8 10 12 14 16 18
These values are in doubleTable:
0 2.14 4.28 6.42 8.56 10.7 12.84 14.98 17.12 19.26

Adding 5 to each element of intTable and doubleTable.
These values are in intTable:
5 7 9 11 13 15 17 19 21 23
These values are in doubleTable:
5 7.14 9.28 11.42 13.56 15.7 17.84 19.98 22.12 24.26

Incrementing each element of intTable and doubleTable.
These values are in intTable:
6 8 10 12 14 16 18 20 22 24
These values are in doubleTable:
6 8.14 10.28 12.42 14.56 16.7 18.84 20.98 23.12 25.26
```

Class Templates and Inheritance

Inheritance can easily be applied to class templates. For example, in the following template, `SearchableVector` is derived from the `SimpleVector` class.

Contents of `SearchableVector.h`

```
#ifndef SEARCHABLEVECTOR_H
#define SEARCHABLEVECTOR_H
#include "SimpleVector.h"

template <typename T>
class SearchableVector : public SimpleVector<T>
{
public:
     // Default constructor
     SearchableVector() : SimpleVector<T>()
          { }

     // Constructor
     SearchableVector(int size) : SimpleVector<T>(size)
          { }

     // Copy constructor
     SearchableVector(const SearchableVector &);

     // Accessor to find an item
     int findItem(const T);
};

//*********************************************************
// Copy constructor                                       *
//*********************************************************

template <typename T>
SearchableVector<T>::SearchableVector(const SearchableVector &obj) :
                     SimpleVector<T>(obj.size())
{
     for(int count = 0; count < this->size(); count++)
          this->operator[](count) = obj[count];
}

//*********************************************************
// findItem function                                      *
// This function searches for item. If item is found      *
// the subscript is returned. Otherwise -1 is returned.   *
//*********************************************************

template <typename T>
int SearchableVector<T>::findItem(const T item)
{
     for (int count = 0; count <= this->size(); count++)
```

```
46      {
47          if (getElementAt(count) == item)
48              return count;
49      }
50      return -1;
51  }
52  #endif
```

This class template defines a searchable version of the `SimpleVector` class. The member function `findItem` accepts an argument and performs a simple linear search to determine whether the argument's value is stored in the array. If the value is found in the array, its subscript is returned. Otherwise, –1 is returned.

Notice each time the name `SimpleVector` is used in the class template, the type parameter `T` is used with it. For example, here is the first line of the class declaration, in line 6, which names `SimpleVector` as the base class:

```
class SearchableVector : public SimpleVector<T>
```

Also, here are the function headers for the class constructors:

```
SearchableVector() : SimpleVector<T>()
SearchableVector(int size) : SimpleVector<T>(size)
```

Because `SimpleVector` is a class template, the type parameter must be passed to it.

Program 16-12 demonstrates the class by storing values in two `SearchableVector` objects, then searching for a specific value in each.

Program 16-12

```
1  // This program demonstrates the SearchableVector template.
2  #include <iostream>
3  #include "SearchableVector.h"
4  using namespace std;
5
6  int main()
7  {
8      const int SIZE = 10;    // Number of elements
9      int count;              // Loop counter
10     int result;             // To hold search results
11
12     // Create two SearchableVector objects.
13     SearchableVector<int> intTable(SIZE);
14     SearchableVector<double> doubleTable(SIZE);
15
16     // Store values in the objects.
17     for (count = 0; count < SIZE; count++)
18     {
19         intTable[count] = (count * 2);
20         doubleTable[count] = (count * 2.14);
21     }
22
23     // Display the values in the objects.
```

(program continues)

Program 16-12 *(continued)*

```
24      cout << "These values are in intTable:\n";
25      for (count = 0; count < SIZE; count++)
26          cout << intTable[count] << " ";
27      cout << endl << endl;
28      cout << "These values are in doubleTable:\n";
29      for (count = 0; count < SIZE; count++)
30          cout << doubleTable[count] << " ";
31      cout << endl;
32
33      // Search for the value 6 in intTable.
34      cout << "\nSearching for 6 in intTable.\n";
35      result = intTable.findItem(6);
36      if (result == -1)
37          cout << "6 was not found in intTable.\n";
38      else
39          cout << "6 was found at subscript " << result << endl;
40
41      // Search for the value 12.84 in doubleTable.
42      cout << "\nSearching for 12.84 in doubleTable.\n";
43      result = doubleTable.findItem(12.84);
44      if (result == -1)
45          cout << "12.84 was not found in doubleTable.\n";
46      else
47          cout << "12.84 was found at subscript " << result << endl;
48      return 0;
49  }
```

Program Output

```
These values are in intTable:
0 2 4 6 8 10 12 14 16 18

These values are in doubleTable:
0 2.14 4.28 6.42 8.56 10.7 12.84 14.98 17.12 19.26

Searching for 6 in intTable.
6 was found at subscript 3

Searching for 12.84 in doubleTable.
12.84 was found at subscript 6
```

The `SearchableVector` class demonstrates that a class template may be derived from another class template. In addition, class templates may be derived from ordinary classes, and ordinary classes may be derived from class templates.

Specialized Templates

Suppose you have a template that works for all data types but one. For example, the `SimpleVector` and `SearchableVector` classes work well with numeric data, and even character data. But they will not work with C-strings. Situations like this require the use of *specialized templates*. A specialized template is one that is designed to work with a specific data

type. In the declaration, the actual data type is used instead of a type parameter. For example, the declaration of a specialized version of the `SimpleVector` class might start like this:

```
class SimpleVector<char *>
```

The compiler would know that this version of the `SimpleVector` class is intended for the `char *` data type. Anytime an object is defined of the type `SimpleVector <char *>`, the compiler will use this template to generate the code.



Checkpoint

16.10 Suppose your program uses a class template named `List`, which is defined as

```
template<typename T>
class List
{
     // Members are declared here...
};
```

Give an example of how you would use `int` as the data type in the definition of a `List` object. (Assume the class has a default constructor.)

16.11 As the following `Rectangle` class is written, the `width` and `length` members are `double`s. Rewrite the class as a template that will accept any data type for these members.

```
class Rectangle
{
     private:
          double width;
          double length;
     public:
          void setData(double w, double l)
               { width = w; length = l;}
          double getWidth()
               { return width; }
          double getLength()
               { return length; }
          double getArea()
               { return width * length; }
};
```

Review Questions and Exercises

Short Answer

1. What is a throw point?
2. What is an exception handler?
3. Explain the difference between a try block and a catch block.
4. What happens if an exception is thrown, but not caught?
5. What is “unwinding the stack”?

6. What happens if an exception is thrown by a class's member function?
7. How do you prevent a program from halting when the `new` operator fails to allocate memory?
8. Why is it more convenient to write a function template than a series of overloaded functions?
9. Why must you be careful when writing a function template that uses operators such as `[]` with its parameters?

Fill-in-the-Blank

10. The line containing a `throw` statement is known as the _____.
11. The _____ block contains code that directly or indirectly might cause an exception to be thrown.
12. The _____ block handles an exception.
13. When writing function or class templates, you use a(n) _____ to specify a generic data type.
14. The beginning of a template is marked by a(n) _____.
15. When defining objects of class templates, the _____ you wish to pass into the type parameter must be specified.
16. A(n) _____ template works with a specific data type.

Algorithm Workbench

17. Write a function that searches a numeric array for a specified value. The function should return the subscript of the element containing the value if it is found in the array. If the value is not found, the function should throw an exception.
18. Write a function that dynamically allocates a block of memory and returns a `char` pointer to the block. The function should take an integer argument that is the amount of memory to be allocated. If the `new` operator cannot allocate the memory, the function should return a null pointer.
19. Make the function you wrote in Question 17 a template.
20. Write a template for a function that displays the contents of an array of any type.

True or False

21. T F There can be only one catch block in a program.
22. T F When an exception is thrown, but not caught, the program ignores the error.
23. T F Data may be passed with an exception by storing it in members of an exception class.
24. T F Once an exception has been thrown, it is not possible for the program to jump back to the throw point.
25. T F All type parameters defined in a function template must appear at least once in the function parameter list.
26. T F The compiler creates an instance of a function template in memory as soon as it encounters the template.

27. T F A class object passed to a function template must overload any operators used on the class object by the template.
28. T F Only one generic type may be used with a template.
29. T F In the function template definition, it is not necessary to use each type parameter declared in the template prefix.
30. T F It is possible to overload two function templates.
31. T F It is possible to overload a function template and an ordinary (nontemplate) function.
32. T F A class template may not be derived from another class template.
33. T F A class template may not be used as a base class.
34. T F Specialized templates work with a specific data type.

Find the Error

Each of the following declarations or code segments has errors. Locate as many as possible.

35.
```
catch
{
    quotient = divide(num1, num2);
    cout << "The quotient is " << quotient << endl;
}
try (string exceptionString)
{
    cout << exceptionString;
}
```

36.
```
try
{
    quotient = divide(num1, num2);
}
cout << "The quotient is " << quotient << endl;
catch (string exceptionString)
{
    cout << exceptionString;
}
```

37.
```
template <typename T>
T square(T number)
{
    return T * T;
}
```

38.
```
template <typename T>
int square(int number)
{
    return number * number;
}
```

39.
```
template <typename T1, typename T2>
T1 sum(T1 x, T1 y)
{
    return x + y;
}
```

40. Assume the following definition appears in a program that uses the `SimpleVector` class template presented in this chapter.

```
int <SimpleVector> array(25);
```

41. Assume the following statement appears in a program that has defined `valueSet` as an object of the `SimpleVector` class presented in this chapter. Assume `valueSet` is a `vector` of `int`s, and has 20 elements.

```
cout << valueSet<int>[2] << endl;
```

Programming Challenges

1. **Date Exceptions**

   Modify the `Date` class you wrote for Programming Challenge 1 of Chapter 13 (Date). The class should implement the following exception classes:

`InvalidDay`	Throw when an invalid day (< 1 or > 31) is passed to the class.
`InvalidMonth`	Throw when an invalid month (< 1 or > 12) is passed to the class.

   Demonstrate the class in a driver program.

2. **Time Format Exceptions**

   Modify the `MilTime` class you created for Programming Challenge 4 of Chapter 15 (Time Format). The class should implement the following exceptions:

`BadHour`	Throw when an invalid hour (< 0 or > 2359) is passed to the class.
`BadSeconds`	Throw when an invalid number of seconds (< 0 or > 59) is passed to the class.

   Demonstrate the class in a driver program.

3. **Minimum/Maximum Templates**

   Write templates for the two functions `minimum` and `maximum`. The `minimum` function should accept two arguments and return the value of the argument that is the lesser of the two. The `maximum` function should accept two arguments and return the value of the argument that is the greater of the two. Design a simple driver program that demonstrates the templates with various data types.

4. **Absolute Value Template**

   Write a function template that accepts an argument and returns its absolute value. The absolute value of a number is its value with no sign. For example, the absolute value of –5 is 5, and the absolute value of 2 is 2. Test the template in a simple driver program.

5. **Total Template**

   Write a template for a function called `total`. The function should keep a running total of values entered by the user, then return the total. The argument sent into the function should be the number of values the function is to read. Test the template in a simple driver program that sends values of various types as arguments and displays the results.

6. `IntArray` Class Exception

Chapter 14 presented an `IntArray` class that dynamically creates an array of integers and performs bounds checking on the array. If an invalid subscript is used with the class, it displays an error message and aborts the program. Modify the class so it throws an exception instead.

7. `TestScores` Class

Write a class named `TestScores`. The class constructor should accept an array of test scores as its argument. The class should have a member function that returns the average of the test scores. If any test score in the array is negative or greater than 100, the class should throw an exception. Demonstrate the class in a program.

8. `SimpleVector` Modification

Modify the `SimpleVector` class template presented in this chapter to include the member functions `push_back` and `pop_back`. The `push_back` function should accept an argument and insert its value at the end of the array. The `pop_back` function should accept no argument and remove the last element from the array. Test the class with a driver program.

9. `SearchableVector` Modification

Modify the `SearchableVector` class template presented in this chapter so it performs a binary search instead of a linear search. Test the template in a driver program.

10. `SortableVector` Class Template

Write a class template named `SortableVector`. The class should be derived from the `SimpleVector` class presented in this chapter. It should have a member function that sorts the array elements in ascending order. (Use the sorting algorithm of your choice.) Test the template in a driver program.

11. Inheritance Modification

Assuming you have completed Programming Challenges 9 and 10, modify the inheritance hierarchy of the `SearchableVector` class template so it is derived from the `SortableVector` class instead of the `SimpleVector` class. Implement a member function named `sortAndSearch`, both a sort and a binary search.

12. Specialized Templates

In this chapter, the section *Specialized Templates* within Section 16.4 describes how to design templates that are specialized for one particular data type. The section introduces a method for specializing a version of the `SimpleVector` class template so it will work with strings. Complete the specialization for both the `SimpleVector` and `SearchableVector` templates. Demonstrate them with a simple driver program.



VideoNote
Solving the Exception Project Problem

13. Exception Project

This assignment assumes you have completed Programming Challenge 1 of Chapter 15 (`Employee` and `ProductionWorker` Classes). Modify the `Employee` and `ProductionWorker` classes so they throw exceptions when the following errors occur:

- The `Employee` class should throw an exception named `InvalidEmployeeNumber` when it receives an employee number that is less than 0 or greater than 9999.
- The `ProductionWorker` class should throw an exception named `InvalidShift` when it receives an invalid shift.
- The `ProductionWorker` class should throw an exception named `InvalidPayRate` when it receives a negative number for the hourly pay rate.

Write a driver program that demonstrates how each of these exception conditions works.